

Hashing, HMAC & Collision Resistance, Worked Out by Hand

A JWT's integrity rests on a keyed hash. We derive why a collision needs $2^{n/2}$ work, brute-force the numbers for 64-, 128-, and 256-bit digests, and show why truncation is fatal.

What this document does. An MCP server trusts a JWT because its HMAC signature can't be forged without the key. The strength of that guarantee is a number of bits. This document derives the two attack costs — preimage (2^n) and collision ($2^{n/2}$, the birthday bound) — and computes brute-force times for 64, 128, and 256-bit digests, showing exactly why truncating a hash is a security cliff.

Concepts covered: hash output bits · preimage vs collision resistance · the birthday bound $2^{n/2}$ · brute-force time at a fixed hash rate · HMAC integrity · the truncation cliff.

Internal learning material. Numbers reproducible from the appendix code. v1.0

1 The setup: a keyed hash protects the token

A JWT is `header.payload.signature`, where `signature = HMACK(header.payload)` for a secret key K . The server recomputes the HMAC and compares. To forge a token an attacker must either find an input hashing to a target (preimage) or two inputs that collide — without K . The cost is set by the digest size n bits. SHA-256 gives $n = 256$.

Intuition

A cryptographic hash is a one-way fingerprint: easy to compute forward, infeasible to invert. HMAC mixes a secret key in, so only someone with K can produce a valid fingerprint — that is what makes the signature unforgeable and the token tamper-evident. “How unforgeable” is literally “how many guesses,” and that is 2 to a power.

2 Step 1 — Two attack costs

Preimage (find any input with a *given* digest): you must search the whole output space,

$$\text{work}_{\text{preimage}} \approx 2^n.$$

Collision (find *any two* inputs with the *same* digest): the birthday effect (Module 1.0) means you only need

$$\text{work}_{\text{collision}} \approx 2^{n/2}.$$

The square-root speed-up is why we care about $n/2$: a 256-bit hash gives 128-bit collision resistance, not 256.

Mini-example — this operation in isolation

Why $2^{n/2}$ and not 2^n ? After hashing m random inputs there are $\binom{m}{2} \approx m^2/2$ pairs, each colliding with probability 2^{-n} , so the expected collisions cross 1 near $m \approx 2^{n/2}$. Same birthday argument as request-ID clashes — collisions are about *pairs*, so the work is the

square root of the space.

3 Step 2 — Brute force in wall-clock time

Assume a very fast adversary computing $H = 10^{12}$ hashes/second (a trillion/s). Collision time is $2^{n/2}/H$ seconds:

digest n	collision work $2^{n/2}$	at $10^{12}/s$	verdict
64	$2^{32} = 4.3 \times 10^9$	0.004 s	trivially broken
128	$2^{64} = 1.8 \times 10^{19}$	213 days	weak, brute-feasible
256	$2^{128} = 3.4 \times 10^{38}$	1.1×10^{19} yr	infeasible

A full SHA-256 ($n = 256$) needs $\sim 10^{19}$ years to collide — vastly longer than the age of the universe. A 64-bit digest falls in *milliseconds*.

4 Step 3 — The truncation cliff

Each bit you drop halves the collision space. Truncating SHA-256 to 64 bits to “save space” moves collision work from 2^{128} to 2^{32} — a factor of $2^{96} \approx 8 \times 10^{28}$ weaker. Security in bits is linear in n , but the *work* is exponential, so a modest-looking truncation is catastrophic.

Watch out

Never truncate a security-bearing hash, and never roll your own comparison. Use the full digest, and compare signatures in *constant time* (`hmac.compare_digest`) — a byte-by-byte early-exit compare leaks timing that recovers the signature one byte at a time, sidestepping the $2^{n/2}$ math entirely. The strongest hash loses to a sloppy `==`.

5 Step 4 — HMAC also pins identity (audience)

A valid signature proves the token wasn’t *tampered* with; it does not prove it was meant for *your* server. That is the **aud** (audience) claim, checked *after* the signature verifies. A signature-valid token with the wrong audience is the confused-deputy gap — correct cryptography, wrong recipient. The hash math secures integrity; the claims secure intent.

6 The argument, at a glance

#	Idea	Formula	Value
1	Preimage work	2^n	2^{256}
1	Collision work	$2^{n/2}$	2^{128}
2	Collision time	$2^{n/2}/H$	10^{19} yr (SHA-256)
2	64-bit collision	$2^{32}/H$	0.004 s
3	Truncation penalty	$2^{(128-32)}$	$8 \times 10^{28} \times$ weaker

7 Equation cheat-sheet

Preimage resistance	$\approx 2^n$
Collision resistance	$\approx 2^{n/2}$ (birthday)
Brute-force time	$2^{n/2}/H$ seconds
Bits after truncation	$n' \Rightarrow \text{collision } 2^{n'/2}$
HMAC	$\text{sig} = \text{HMAC}_K(\text{msg})$, constant-time compare

A Reference implementation

```
SECONDS_PER_YEAR = 365.25 * 24 * 3600
H = 1e12                                # hashes per second

def collision_time_years(n):
    work = 2 ** (n / 2)
    return work / H / SECONDS_PER_YEAR

print(2**32 / H)                        # 0.0043 s (n=64)
print(round(2**64 / H / 86400))         # 213 days (n=128)
print(f"{collision_time_years(256):.1e}") # 1.1e+19 years
```

— end of document —